

Affinity metrics

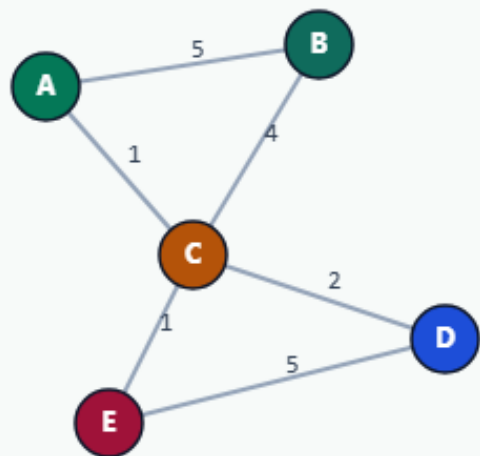
Before you merge cells or coarsen a netlist, you need a score that says these two should stay together

Start with a tiny weighted graph

AFFINITY METRICS

Start with a tiny weighted graph

Step 1 / 5 · raw-graph · module01-01-affinity-metrics



Explanation

Five cells A-E with edge weights. Heavy edges A-B and D-E both weigh 5; the bridge through C is weaker. Affinity will decide which pairs want to cluster first.

- Nodes = cells / clusters to group
- Edge weight = how strongly a pair should stay together
- Reference instance used across all clustering labs

Graph: 5 nodes, 6 edges
Heaviest edges: A-B=5, D-E=5

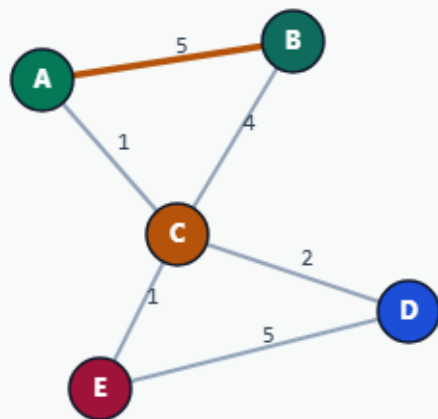
Start with a tiny weighted graph

Edge-weight affinity ranks pairs

AFFINITY METRICS

Edge-weight affinity ranks pairs

Step 2 / 5 · edge-rank · module01-01-affinity-metrics



Explanation

Pure edge-weight affinity just sorts existing edges. A-B and D-E tie at 5; tie-break puts A-B first. This ranking is the simplest merge priority.

- $\text{score}(u,v) = w(u,v)$
- Sort descending by score
- Top pair becomes the first merge candidate

1. A-B @ 5
2. D-E @ 5
3. B-C @ 4
4. C-D @ 2
5. A-C @ 1
6. C-E @ 1

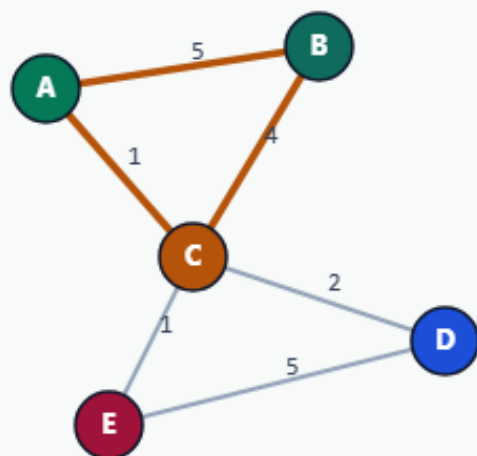
Edge-weight affinity ranks pairs

Shared neighbors boost A-B

AFFINITY METRICS

Shared neighbors boost A-B

Step 3 / 5 · shared-idea · module01-01-affinity-metrics



Explanation

A and B already share edge weight 5. They also share neighbor C. Shared-neighbor affinity adds $\min(w(A,C), w(B,C)) = \min(1, 4) = 1$, so A-B rises to 6.

- Direct edge still counts
- Plus support through common neighbors
- Encourages dense triangles to cluster

$w(A,B)=5$
shared via C: $\min(1,4)=1$
 $\text{score}(A,B)=6$

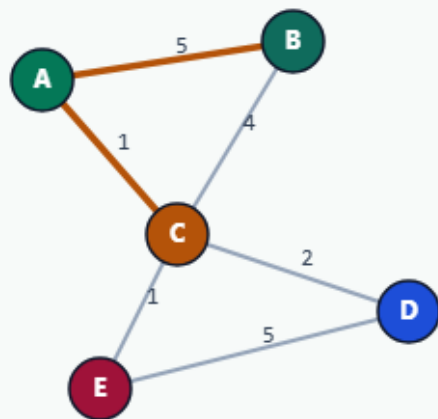
Shared neighbors boost A-B

Shared ranking reshuffles the list

AFFINITY METRICS

Shared ranking reshuffles the list

Step 4 / 5 · shared-rank · module01-01-affinity-metrics



Explanation

Under shared-neighbor scoring, A-B and D-E lead at 6. Weak edge A-C jumps to 5 because of B. Non-edges like B-D can appear from shared support alone.

- A-C: was 1 → now 5
- Invented pairs (e.g. B-D @ 2) show transitive pull
- Same idea feeds greedy merge priorities

1. A-B @ 6
2. D-E @ 6
3. A-C @ 5
4. B-C @ 5
5. C-D @ 3
6. C-E @ 3

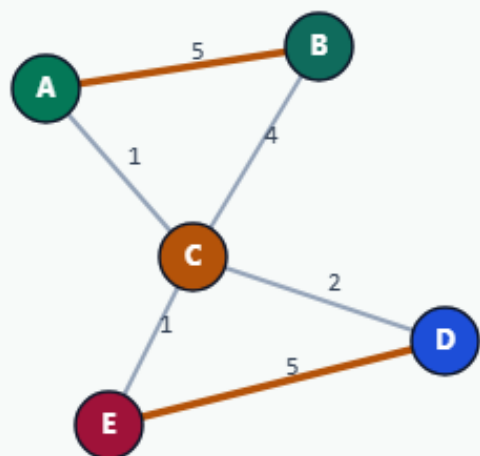
Shared ranking reshuffles the list

Why affinity matters in EDA

AFFINITY METRICS

Why affinity matters in EDA

Step 5 / 5 · takeaway · module01-01-affinity-metrics



Explanation

Affinity is the objective's local preference: which cells belong together before global constraints. Bad affinity → bad merges → harder refinement later.

- Edge weight = connectivity / nets shared
- Shared neighbors $\approx$ denser local groups
- Next labs consume this ranking to merge

Edge top: A-B @ 5

Shared top: A-B @ 6

Teaching graph stays fixed for goldens

Why affinity matters in EDA

Browser lab track

- In the browser lab track, open the affinity metrics lab, load the starter example
- Clear the ten challenges when you're ready

Implement track

- In the implement track, load the tiny weighted graph and print both rankings
- Match the goldens: edge weight tops at five for A-B; shared-neighbor lifts A-B to six
- Run the unit tests from the course common folder so your scores stay honest

Implement track — try these

```
# pwd — print working directory (where am I?)  
pwd
```

```
# ls examples — confirm the starter graph is here  
ls examples
```

```
# print both affinity rankings  
export PYTHONPATH=../common
```

```
python ../common/solvers.py examples/tiny_graph.json --mode affinity
```

Pitfalls to watch

- Watch for mixing directed and undirected assumptions, or double-counting a neighbor
- Don't invent pairs with zero connection unless your API says so
- And don't let a later merge engine silently use a different affinity than the one you

Your turn

- Finish the checklist for at least one track
- Match both golden tables, then explain why A–C climbs under shared-neighbor scoring
- Take the short quiz